



Running PostgreSQL on Kubernetes in Production

Day 2 Operations with CloudNativePG

Michael Raeck

ContainerDays Hamburg · September 2026





\$whoami

Michael Raeck

"Hands on" CTO, Paladyn · venture builder (Woonzorgweb, ~150k users/month)

Independent Platform Engineer · Vantralis

eucloudcost.com · Personal Project, Comparison of Eu Cloud Providers, open Data Source

Daily Work

Backend Development, Kubernetes, GitOps & CloudNativePG in ISO 27001-certified environments

100+ CNPG databases



CloudNativePG (CNPG)

A Kubernetes operator that runs PostgreSQL. Not a distributed database.

What it is: a controller that owns Pods, PVCs and Services directly, No StatefulSet, no Patroni, no external DCS. CNCF Incubating.

What it is not: **one primary, N read-only replicas**. No multi-master, no active-active, no writes at two sites.

1.30.0

current stable, 29 June 2026

1.29.x

supported until **29 Sep 2026**, four weeks from now

1.28 / 1.27 / 1.26

end of life

PostgreSQL 14–18

18.4 at 1.30.0's release · **18.6** current (18.5 never shipped)

Kubernetes 1.34–1.36

⚠ **Three CVEs.** CVE-2026-44477 , CVSS 9.4, **metrics exporter** (fixed 1.29.1 / 1.28.3). CVE-2026-55769 privilege escalation via the public schema, and CVE-2026-55765 cleartext passwords on CREATE/ALTER ROLE , both fixed in **1.30.0**, backported to 1.29.2 / 1.28.4.



The Operator Is the Easy Part

One CR gets you a cluster. Everything after you somehow need to measure.

Storage: IOPS is critical for DB Operations,

a 1 TB IOT Data DB with 5000 R/W operations per seconds, operate differently with 500 vs 5.000 vs 50.000 IOPS

a Sonarqube DB with 10 Users in your Cluster is maybe fine with 1.000 IOPS

Object store: If your object Store is attached with 100 MB/s to your Datacenter:

can lead to Restore times of Hours

Topology: How you want to build it, what is your Scenario, Edge Clusters, Pilot Light, Warm Standby ?

We will look into Demos to see if there are good Options.



Let's deploy

One Cluster CR,
three instances





Demo 1: Deploy

```
spec:
  cluster:
    name: hafen
  instances: 2
  type: rw # rw = primary; ro = read-only replicas
  pgbouncer:
    poolMode: transaction
    parameters:
      max_client_conn: "1000"
      default_pool_size: "25"
> kubectl apply -f 01b-pooler.yaml
pooler.postgresql.cnpg.io/hafen-pooler-rw created
> kubectl -n hafen rollout status deploy/hafen-pooler-rw --timeout=180s
Waiting for deployment "hafen-pooler-rw" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "hafen-pooler-rw" rollout to finish: 1 of 2 updated replicas are available...
deployment "hafen-pooler-rw" successfully rolled out
> kubectl -n hafen get pods -l cnpg.io/poolerName=hafen-pooler-rw
NAME                                READY   STATUS    RESTARTS   AGE
hafen-pooler-rw-9b959658d-lgwm7     1/1     Running   0           11s
hafen-pooler-rw-9b959658d-n7fmx     1/1     Running   0           11s
> # Connect through the pooler, not the database. Transaction-mode pooling.
> PGPW=$(kubectl -n hafen get secret hafen-app -o jsonpath="{.data.password}" | base64 -d)
> PRIMARY=$(kubectl -n hafen get pod -l cnpg.io/cluster=hafen,cnpg.io/instanceRole=primary -o jsonpath="{.items[0].metadata.name}")
> kubectl -n hafen exec $PRIMARY -c postgres -- env PGPASSWORD=$PGPW psql -h hafen-pooler-rw -U hafenmeister hafen -c "S
ELECT count(*) FROM containers;"
count
-----
4
(1 row)
```



Day 0 Decides Your Day 2

You just watched me deploy `storage: 2Gi` with no storageClass.

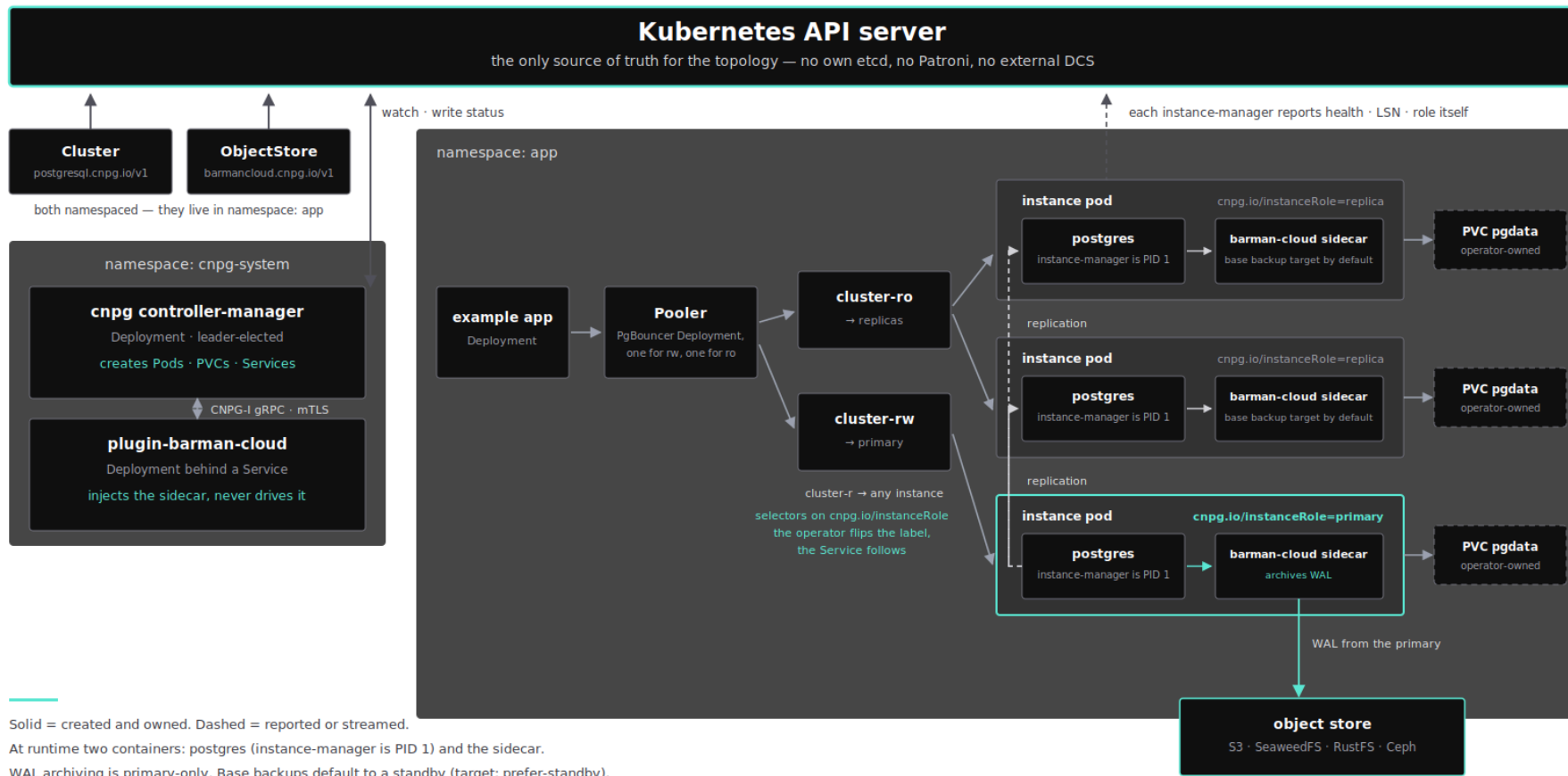
Volume	Published performance
OVH High Speed Gen2	30 IOPS/GB → max 20,000 · 0.5 MB/s/GB → max 320 MB/s
Scaleway Block Storage	up to 15,000 IOPS , "subject to fluctuation during peak load"
Hetzner Cloud Volumes	not published . "fast (SSD based)"
AWS gp3	3,000 IOPS + 125 MiB/s baseline, no bursting
AWS gp2	3 IOPS/GiB → 100 GiB = 300 IOPS, bursts, then 5 h to refill credits
Netways performance-optimized-2	base 5000 IOPS , burst 10000
StackIt	up to 60000
OnPrem	only you know

IOPS is often a function of volume size: check how your volume Size correlates with your IOPS

✓ `allowVolumeExpansion`



The Whole Architecture





Let's back it up

An ObjectStore and a sidecar

Wire the bucket · a sidecar appears · one backup
now, one every night





Demo 2: ObjectStore + Backup

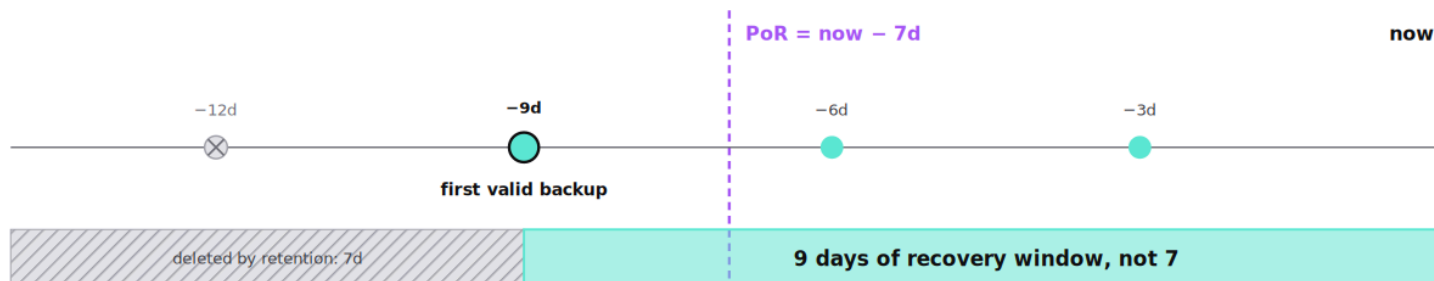
```
scheduledbackup.postgresql.cnpg.io/hafen-nightly created
> timeout 60 kubectl -n hafen get backup -w
NAME                AGE    CLUSTER  METHOD  PHASE    ERROR
hafen-backup-initial 2s     hafen    plugin started
hafen-backup-initial 10s    hafen    plugin completed
> kubectl -n hafen wait backup/hafen-backup-initial --for=jsonpath="{.status.phase}"=completed --timeout=300s
backup.postgresql.cnpg.io/hafen-backup-initial condition met
> kubectl -n hafen get backup,scheduledbackup
NAME                AGE    CLUSTER  METHOD  PHASE    ERROR
backup.postgresql.cnpg.io/hafen-backup-initial 70s    hafen    plugin completed

NAME                AGE    CLUSTER  LAST BACKUP
scheduledbackup.postgresql.cnpg.io/hafen-nightly 70s    hafen
> # The ObjectStore reports the window you actually have, not the one you configured.
> kubectl -n hafen get objectstore hafen-store -o jsonpath="{.status.serverRecoveryWindow}" | jq
{
  "hafen": {
    "firstRecoverabilityPoint": "2026-08-26T21:52:07Z",
    "lastSuccessfulBackupTime": "2026-08-26T21:52:07Z"
  }
}
> # Base backup + WAL are in S3 (SeaweedFS). This is what PITR + DR will reuse.
> kubectl -n seaweedfs run s3ls --rm -i --restart=Never --image=amazon/aws-cli:2.36.31 --env=AWS_ACCESS_KEY_ID=hafenmeister --env=AWS_SECRET_ACCESS_KEY=schuppen52rules --env=AWS_DEFAULT_REGION=us-east-1 -- --endpoint-url http://seaweedfs:9000 s3 ls --recursive s3://hafen-backups/
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
2026-08-26 21:52:07      1433 hafen/base/20260826T215159/backup.info
2026-08-26 21:52:07    4079895 hafen/base/20260826T215159/data.tar.gz
2026-08-26 21:51:20    91541 hafen/wals/0000000100000000/000000010000000000000006.gz
```



Retention Is Not Your Recovery Window

Retention deletes **backups**, not time. The oldest *surviving* base backup sets your real PoR.



Retention **7d** + backups every **3d** $\Rightarrow$ real window **9d**. Change the schedule, you change the window.

paladyn

Let's break it
On purpose.





Demo 3: Point-in-Time Recovery

```
SET
SET
SET
SET
SET
set_config
-----
```

(1 row)

```
SET
SET
SET
SET
COPY 4
setval
-----
```

4
(1 row)

> # Production is whole again. THIS is what the restore was for.

> kubectl -n hafen exec \$PRIMARY -c postgres -- psql -U postgres -d hafen -c "SELECT ship, cargo, weight_kg FROM containers ORDER BY weight_kg DESC LIMIT 5;"

ship	cargo	weight_kg
Klaus Stoertebeker	Elbtower-Fassade, Rest	43999
Goedeke Michels	U5-Tunnelbohrer, ETA 2040	41250
Cap San Diego	Astra Bier	38400
MS Bleichen	Franzbroetchen	17200

(4 rows)

■

▶ [play this recording — mixxor.github.io/containerdays-2026-cnpg/demo3.html](https://mixxor.github.io/containerdays-2026-cnpg/demo3.html)



PITR: how to recover

There is no in-place recovery. Recovery is **always** a new cluster.

```
bootstrap:
  recovery:
    source: hafen-source
    recoveryTarget:
      targetTime: "2026-09-03 10:42:17+00"
```

Target	Use when
targetTime	you know roughly when it happened
targetLSN	you know exactly where in the WAL
targetXID	you have the offending transaction id
targetName	you called <code>pg_create_restore_point()</code> before the risky change

Restore time = fetch base backup + replay WAL since that backup.



Time to Recovery

count in Network to your TTR.

Database	25 MB/s cross-region	100 MB/s	250 MB/s	500 MB/s same DC
10 GB	7 min	2 min	41 s	20 s
100 GB	68 min	17 min	7 min	3 min
500 GB	5.7 h	85 min	34 min	17 min
1 TB	11.7 h	2.9 h	70 min	35 min

+ all Wal Replays since last Base Backup

gZip should be used, but only swaps Transport for CPU, but this can still be faster

can be tweaked: `ObjectStore` , `data.jobs` and `wal.maxParallel` .

Measure this: The whole Recovery procedures require this



Let's upgrade it

Postgres 17 to 18 in
place





Demo 4: In-Place Major Upgrade, 17 → 18

```
hafen-1 0/12001D48 Primary OK Burstable 1.30.0 hafen-worker2
hafen-2 0/12001D48 Standby (async) OK Burstable 1.30.0 hafen-worker3
hafen-3 0/12001D48 Standby (async) OK Burstable 1.30.0 hafen-worker
```

Plugins status

Name	Version	Status	Reported Operator Capabilities
barman-cloud.cloudnative-pg.io	0.14.0	N/A	Reconciler Hooks, Lifecycle Service, TYPE_INSTANCE_SIDE CAR_INJECTION

```
> # Proof.
> PRIMARY=$(kubectl -n hafen get pod -l cnpg.io/cluster=hafen,cnpg.io/instanceRole=primary -o jsonpath="{.items[0].metadata.name}"); echo $PRIMARY
hafen-1
> kubectl -n hafen exec $PRIMARY -c postgres -- psql -U postgres -c "SHOW server_version;"
server_version
```

```
-----
18.6 (Debian 18.6-1.pgdg13+2)
(1 row)
```

```
> # PG 18 now preserves optimizer stats - but NOT extended statistics. ANALYZE anyway.
> kubectl -n hafen exec $PRIMARY -c postgres -- psql -U postgres -d hafen -c "ANALYZE;"
ANALYZE
> # And every base backup in the bucket is still PG 17 - it cannot bootstrap PG 18.
> kubectl apply -f 04b-post-upgrade-backup.yaml
backup.postgresql.cnpg.io/hafen-backup-pg18 created
> kubectl -n hafen wait backup/hafen-backup-pg18 --for=jsonpath="{.status.phase}"=completed --timeout=600s
backup.postgresql.cnpg.io/hafen-backup-pg18 condition met
> kubectl -n hafen get backup
NAME                AGE    CLUSTER  METHOD  PHASE    ERROR
hafen-backup-initial 6m11s hafen    plugin completed
hafen-backup-pg18    18s    hafen    plugin completed
```



A Major Upgrade will lead to replicas being recreated based on the primary

Database	Writes fail	No HA 3 inst	No HA 5 inst
10 GB	5 min	9 min	12 min
100 GB	5 min	39 min	1.2 h
500 GB	5 min	2.9 h	5.8 h
1 TB	5 min	5.8 h	11.4 h

100 MB/s volume · 500 tables × 12 columns

Always Backup before a Major migration

✓ `pg_upgrade --link` hard-links, this scales with your **schema**, not your bytes. That's why R/W downtime is static

✗ **Replica rebuilds** are `pg_basebackup` from the **primary**, one at a time. `-ro` has no endpoints until they finish.

Same OS distribution, always. `17.6-bookworm` → `18.6-trixie` is rejected: different glibc, different collation order, indexes sorted by rules that no longer apply.



Let's build a second site

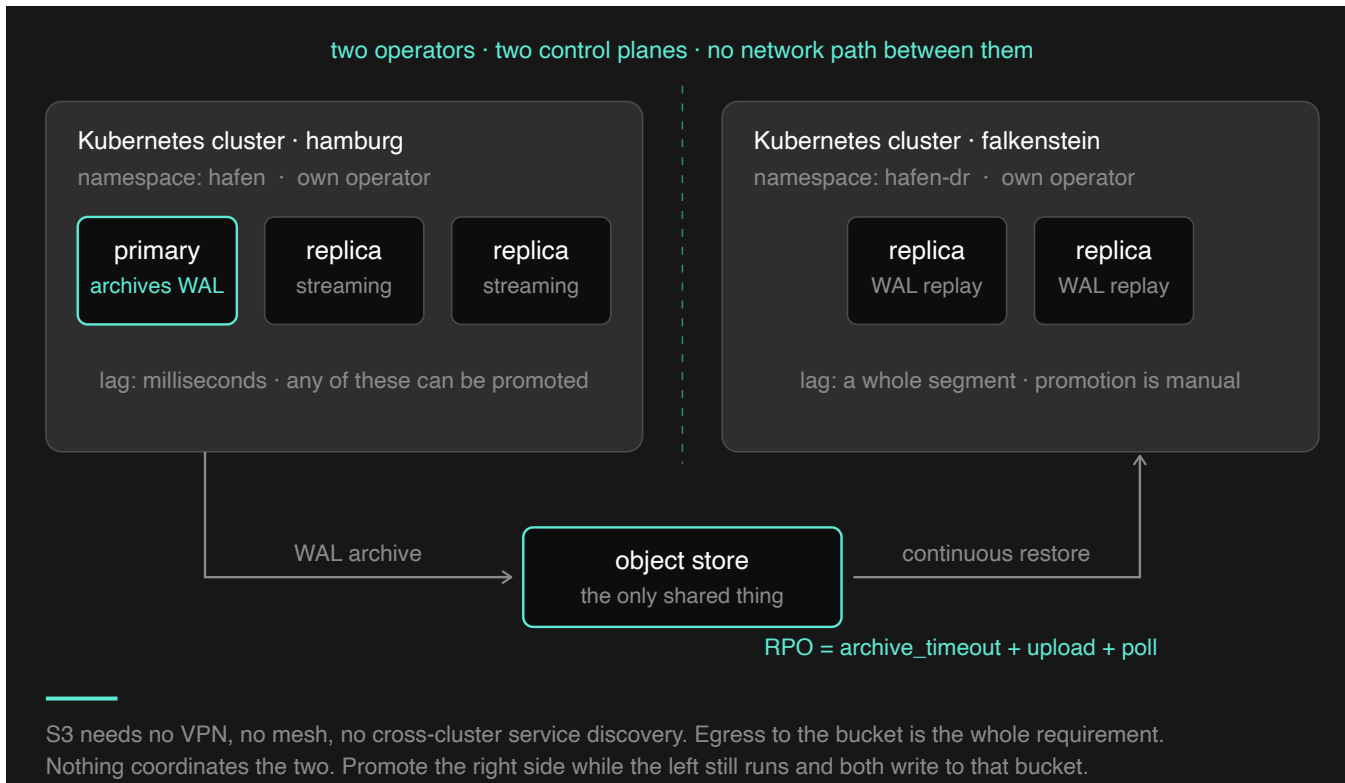
A replica cluster, on
the Edge





Why the Edge Site Is a Second Cluster

You cannot pin a primary. So you draw a cluster boundary instead.



One pod template per cluster · `spec.affinity` is cluster-wide · membership **is** eligibility



Demo 5: Replica Cluster

Streaming Replication status

Replication Slots Enabled

Name	Sent LSN	Write LSN	Flush LSN	Replay LSN	Write Lag	Flush Lag	Replay Lag	State	Sync State	Sync
c Priority	Replication	Slot								
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	---
hafen-dr-2	0/140000A0	0/140000A0	0/140000A0	0/140000A0	00:00:00	00:00:00	00:00:00	streaming	async	0
	active									

Instances status

Name	Current LSN	Replication role	Status	QoS	Manager Version	Node
-----	-----	-----	-----	---	-----	---
hafen-dr-1	0/140000A0	Primary	OK	BestEffort	1.30.0	hafen-worker
hafen-dr-2	0/140000A0	Standby (async)	OK	BestEffort	1.30.0	hafen-worker2

Plugins status

Name	Version	Status	Reported Operator Capabilities
-----	-----	-----	-----
barman-cloud.cloudnative-pg.io	0.14.0	N/A	Reconciler Hooks, Lifecycle Service, TYPE_INSTANCE_SIDE CAR_INJECTION

```
> # It is a primary now, so it takes writes. A standby would refuse this.
> kubectl -n hafen-dr exec $DRP -c postgres -- psql -U postgres -d hafen -c "CREATE TABLE promoted_in_hamburg (id serial
primary key)"
CREATE TABLE
> kubectl -n hafen-dr exec $DRP -c postgres -- psql -U postgres -d hafen -c "SELECT count(*) FROM containers"
count
-----
4
(1 row)
```



Gotchas

Measure S3, Volumes,... everything!!

Do DR Drills with your biggest DBs

A major upgrade needs a backup on **both** sides

Recovery is always a new cluster

Recovery Window is wider then you think

Day 2 starts on day 0

A **failed** upgrade you revert with `imageName` . A **successful** one already replaced the old directories - and a PG 17 backup cannot bootstrap PG 18.

There is no in-place undo. Practise it.

Object Store CR Retention + Scheduled Backup Interval is your Recovery window

One CR gets you a cluster. and it seems Easy, but its actually a bit OPS heavy



Resources

- cloudnative-pg.io/docs · github.com/cloudnative-pg/cloudnative-pg
- Barman Cloud plugin (CNPG-I): github.com/cloudnative-pg/plugin-barman-cloud
- `ObjectStore` [reference](#), every field on the CR: cloudnative-pg.io/plugin-barman-cloud/docs/object_stores
- [Grafana dashboard ID 20417](#)
- `kubectl krew install cnpg`

Slides, and Demos:

github.com/mixxor/containerdays-2026-cnpg



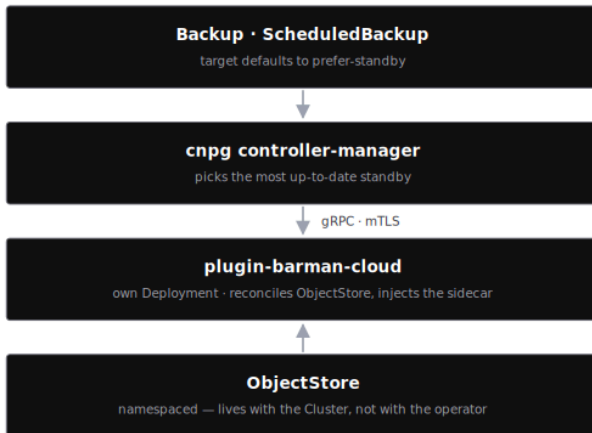
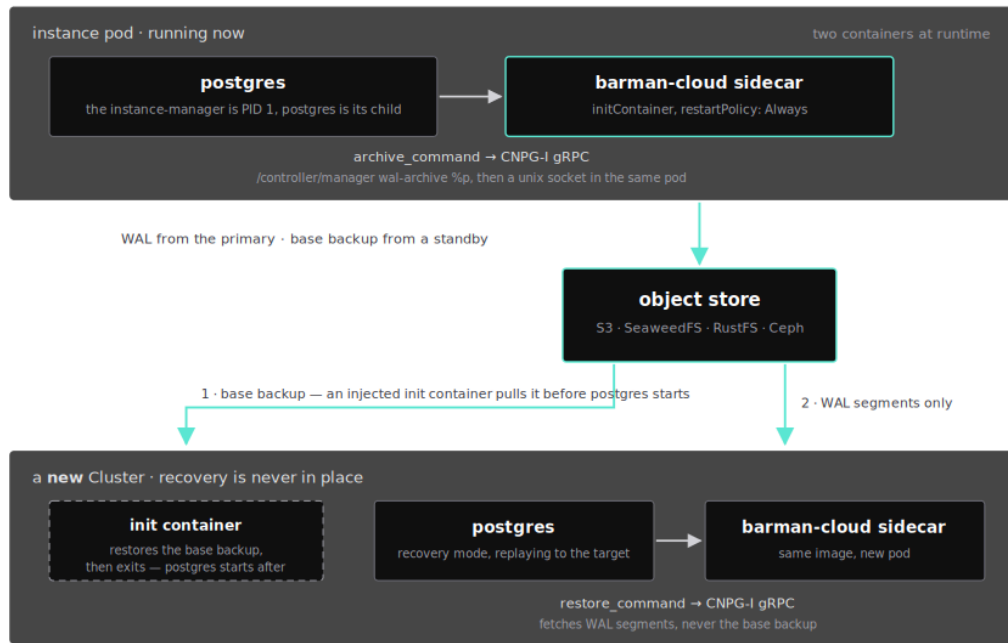
paladın

BACKUP





How Backups Actually Work



The base backup runs in the sidecar, next to the data — on a standby by default, on the primary only as a fallback.

WAL archiving is the exception: primary only.

Credentials stay in the Cluster's namespace.

Why a sidecar and not the operator: **IRSA per cluster · blast radius · version skew**



What You Actually Get at the Edge

Local reads. Not local writes. And two different tools with the same field name.

	Streaming replica same cluster	Standalone replica cluster	Distributed topology
Lag	milliseconds	<code>archive_timeout</code> , 5 min default	same, or streaming
Reads	yes, <code>-ro</code> Service	yes	yes
Writes	no	no	no
Promotion	automatic, on failover	one field, irreversible	demotion → promotion token
Old primary after	rejoins via <code>pg_rewind</code>	must be re-cloned	becomes a replica
Docs say it's for	—	read-only workloads	DR and HA

✗ **A replica cluster never accepts a write.** `CREATE TABLE` fails with *cannot execute in a read-only transaction*, exactly as on any standby. ✓ **Want ~1 min RPO?** Set `archive_timeout: 60s` and pay in forced segment switches. ✓ **Want protection from demo 3?** `replica.minApplyDelay: 8h` — a replica that intentionally lags, so you can catch the missing `WHERE` before it lands.

Chain: segment fills or `archive_timeout` fires → upload → the standby's `restore_command` asks → replay. The first term dominates. Everything else is seconds.



Monitoring

Instance :9187/metrics · Operator :8080/metrics · Grafana dashboard [20417](#)

Category	Metric	Alert threshold
Health	cnpg_collector_up	= 0 for 1m
Health	cnpg_cluster_ready_instances	< expected for 5m
Replication	cnpg_pg_replication_lag	> 30s warn / > 300s crit
WAL	cnpg_collector_pg_wal_archive_status	failed count > 0
Safety	cnpg_pg_database_xid_age	> 300,000,000
Backup	barman_cloud_cloudnative_pg_io_last_available_backup_timestamp	older than 2× your schedule

This is the dashboard from the run you just watched.



Automated Failover

No Patroni. No Stolon. Instance manager + Kubernetes.

Phase 1: detection & shutdown	Phase 2: promotion
Primary readiness probe fails	Leader election among replicas
Controller marks TargetPrimary pending	Chosen replica promotes
Fast shutdown signal → primary	Former primary rejoins via <code>pg_rewind</code>
WAL receivers on replicas stop	Cluster resumes

```
spec:
  instances: 3
  failoverDelay: 10          # seconds, prevents flapping
  switchoverDelay: 60
  postgresql:
    synchronous:
      method: quorum
      number: 1
      failoverQuorum: true   # stable since 1.28
```



Where Do Instances Actually Land?

Spreading is a *hint*, not a guarantee

```
spec:
  affinity:
    enablePodAntiAffinity: true           # default
    podAntiAffinityType: preferred        # default ← a hint
    topologyKey: kubernetes.io/hostname  # default
```

5 instances, 3 worker nodes. What actually happened:

```
hafen-2   hafen-worker3   hafen-4   hafen-worker
hafen-3   hafen-worker     hafen-5   hafen-worker3   ← doubled up
```

- ❌ `podAntiAffinityType: required` → pods **Pending forever** when instances > nodes
- ✅ `spec.topologySpreadConstraints`, top level, expresses `maxSkew`, which anti-affinity cannot

`syncReplicaElectionConstraint` **does not solve this**: it governs *sync replica election* only; an excluded replica can still be promoted.



Quorum Failover Maths

$$R + W > N$$

R	promotable replicas reporting state to the operator
W	replicas acknowledging synchronous commits
N	total potentially synchronous replicas

Cluster	Failure	Failover?	Why
3-node, sync=1	primary only	yes	$R=2, W=1, N=2 \rightarrow 3 > 2$
3-node, sync=1	primary + 1 replica	no	$R=1, W=1, N=2 \rightarrow 2 = 2$
5-node, sync=2	primary + 1 replica	yes	$R=3, W=2, N=4 \rightarrow 5 > 4$

`local` and `off` **bypass the guarantee.** `on`, `remote_write` and `remote_apply` all wait for the synchronous standbys. A session doing `SET synchronous_commit TO local` does not.



Storage

CNPG manages PVCs directly, with no StatefulSet

```
spec:
  storage:    { size: 100Gi, storageClass: premium-ssd }
  walStorage: { size: 20Gi,  storageClass: premium-ssd-wal }
```

StorageClass requirements

- ✓ `WaitForFirstConsumer` for zone-aware binding
- ✓ `allowVolumeExpansion`, your only way out of a full disk
- ✓ `reclaimPolicy: Retain` · SSD / NVMe only

Anti-patterns

- ✗ NFS or HDD without IOPS guarantees
- ✗ `ReadWriteMany` · skipping `WaitForFirstConsumer`

Separate the WAL volume, and size it $\geq 3 \times \text{max_wal_size}$.



Services & Connection Routing

Three Services per cluster. The operator keeps their endpoints correct

Service	Targets	Use
hafen-rw	the current primary only	all writes
hafen-ro	replicas only	read-only queries
hafen-r	every instance, primary included	"any node will do"

`-ro` has no endpoints when every replica is down. `-r` still resolves.

Pooler mode	Best for	Caveat
transaction	web apps	no session state, no prepared statements
session	legacy apps	far lower connection reuse



GitOps with ArgoCD

```
syncPolicy:
  automated:
    prune: false           # NEVER auto-delete a database
    selfHeal: true
  syncOptions:
    - ServerSideApply=true
```

- `prune: false` : a pruned `Cluster` is a deleted database
- **ServerSideApply**: the operator writes to status/spec; client-side apply fights it
- **SealedSecrets / ESO**: never plain Secrets in Git
- **One Application per cluster**: independent lifecycle and blast radius



Alerting on Those Metrics

Seven PrometheusRules ship with the operator; these are the ones worth tuning

```
groups:
- name: cloudnativepg.rules
  rules:
    - alert: CNPGClusterDown
      expr: cnpg_cluster_ready_instances == 0
      labels: { severity: critical }

    - alert: CNPGClusterHADegraded
      expr: cnpg_cluster_ready_instances < cnpg_cluster_instances
      for: 5m
      labels: { severity: warning }

    - alert: CNPGWALArchiveFailing
      expr: cnpg_collector_pg_wal_archive_status{status="failed"} > 0
      for: 5m
      labels: { severity: critical }
```

WAL archive failure is the one to page on. It silently invalidates every future PITR.



Top Production Issues

Issue	Symptom	Fix
WAL disk exhaustion	primary crash-loops	expand PVC; check for stale replication slots
Replica out of sync	WAL gap, crash-loop	delete pod + PVC → operator re-clones via <code>pg_basebackup</code>
NetworkPolicy blocks operator	"cannot extract Pod status"	allow <code>cnpg-system</code> → port 8000/TCP. Since 1.30 that channel is ECDSA-authenticated (not backported), so on ≤ 1.29 keep restricting it
XID wraparound	<code>xid_age > 300M</code>	<code>VACUUM (FREEZE, VERBOSE)</code> on the affected table
HugePages mismatch	bus error, exit 135	<code>huge_pages: try</code> , or <code>hugepages-2Mi ≥ shared_buffers</code>
OOMKill	exit 137, no PG error	memory requests = limits; recheck <code>shared_buffers + work_mem × conns</code>

```
kubectl cnpg status hafen -n hafen
kubectl cnpg report cluster hafen -n hafen --logs # everything, zipped
```